

# what does a computer actually do?

digitalisierung und programmierung · input, session 12

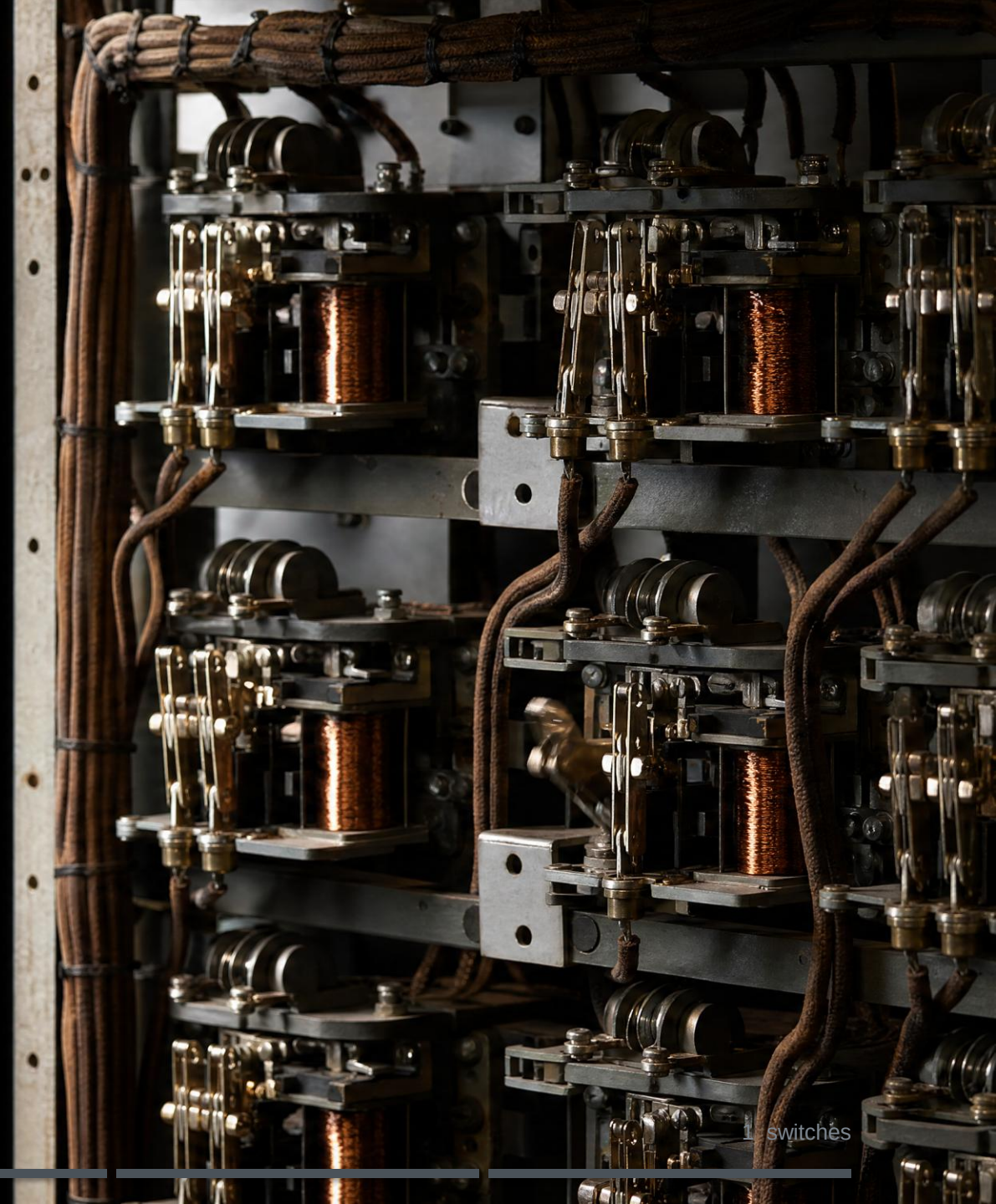
## agenda

- 1 switches
- 2 storing
- 3 adding
- 4 a photo, pixel by pixel
- 5 one gate, two jobs

# switches

what can you build  
out of switches?

image: ai-generated (gpt-image-2)



switches

three things a computer does with bits

store them

three things a computer does with bits

store them

move them

three things a computer does with bits

store them

move them

change them by rules

the whole toolbox

and

| a | b | out |
|---|---|-----|
| 0 | 0 | 0   |
| 0 | 1 | 0   |
| 1 | 0 | 0   |
| 1 | 1 | 1   |

1 only if both are 1

or

| a | b | out |
|---|---|-----|
| 0 | 0 | 0   |
| 0 | 1 | 1   |
| 1 | 0 | 1   |
| 1 | 1 | 1   |

1 if at least one is 1

not

| a | out |
|---|-----|
| 0 | 1   |
| 1 | 0   |

flips the bit

xor

| a | b | out |
|---|---|-----|
| 0 | 0 | 0   |
| 0 | 1 | 1   |
| 1 | 0 | 1   |
| 1 | 1 | 0   |

1 if the two differ

two bits in, one bit out, one fixed rule. that is the whole toolbox.



one gate is enough

nand: and, then not

| a | b | out |
|---|---|-----|
| 0 | 0 | 1   |
| 0 | 1 | 1   |
| 1 | 0 | 1   |
| 1 | 1 | 0   |

what a transistor pair does by itself

not a = **nand(a, a)**

feed the same bit in twice

a and b = **not nand(a, b)**

a nand, then a not

a or b = **nand(not a, not b)**

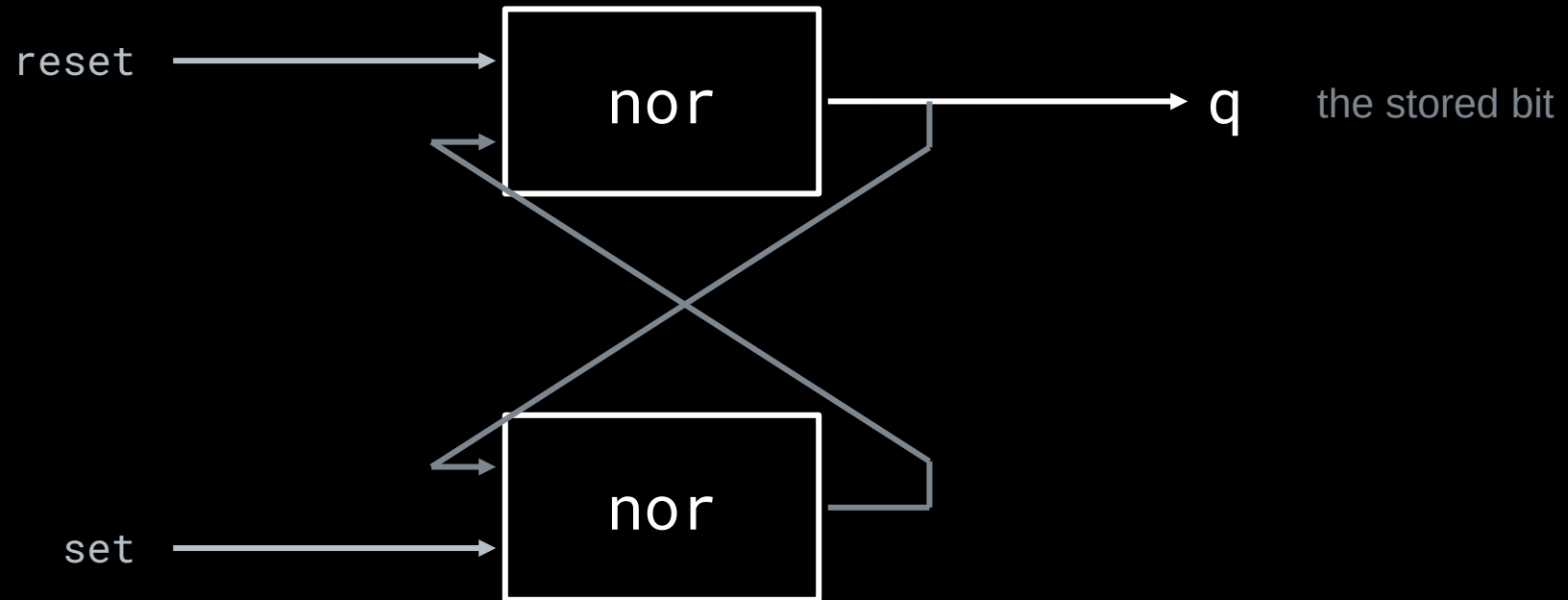
flip both, then nand

**every gate in the box is a few nands. so is your laptop.**

no gate understands anything.  
it just switches.

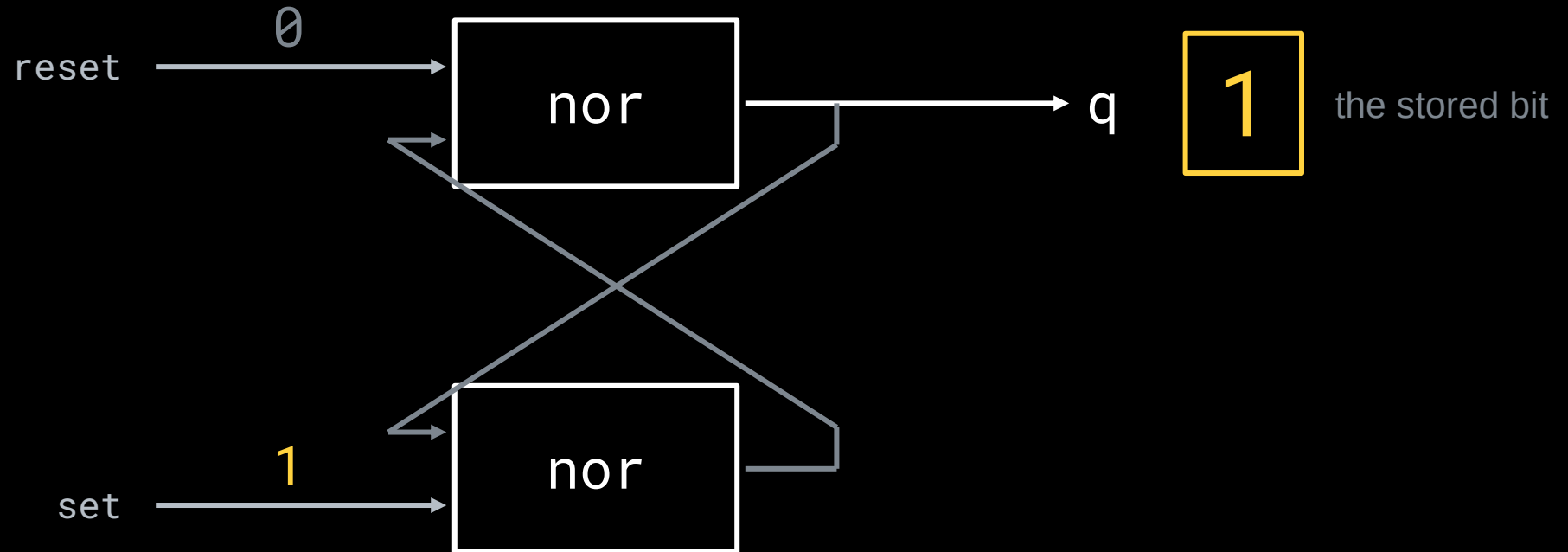
storing: a bit that remembers

two nor gates, wired into each other



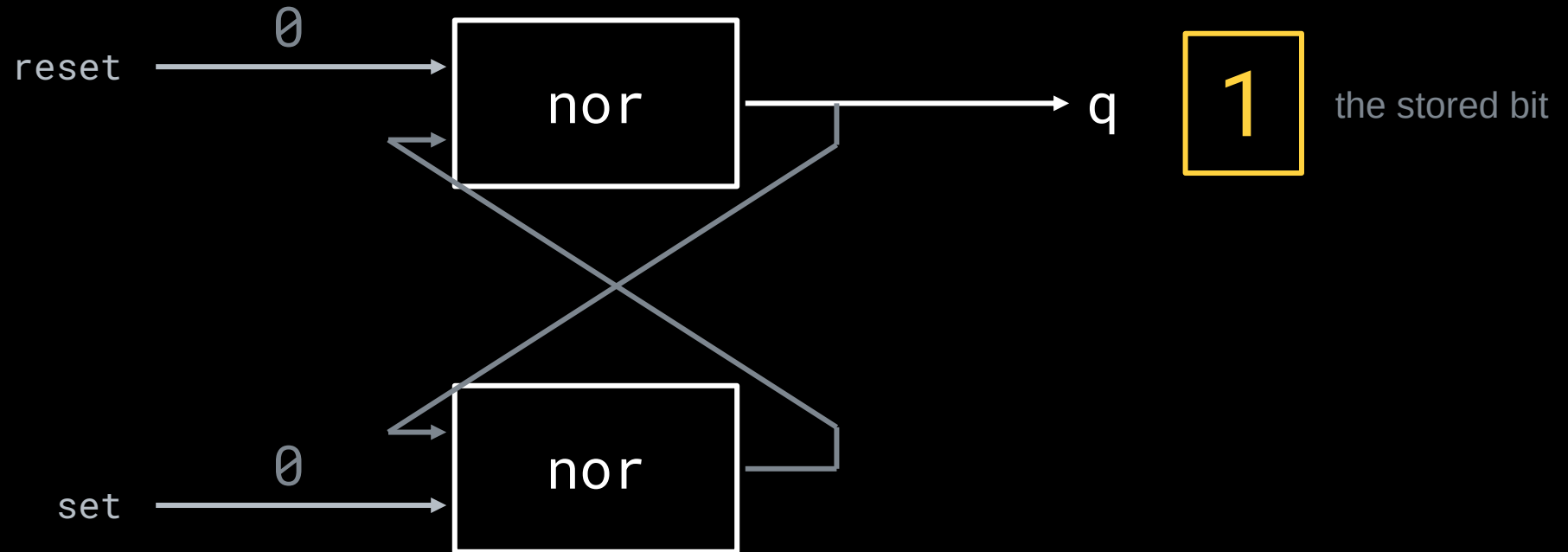
the two outputs feed back into each other's input.

two nor gates, wired into each other



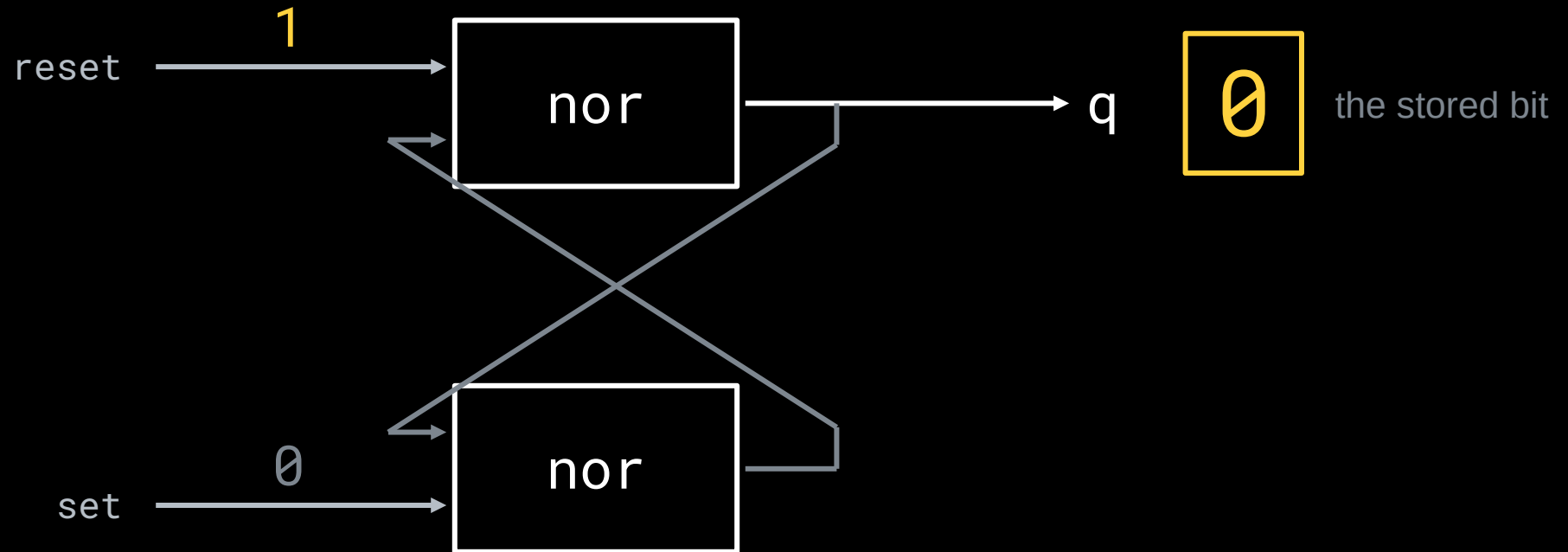
set = 1: q becomes 1.

two nor gates, wired into each other



set back to 0: q stays 1. it remembers.

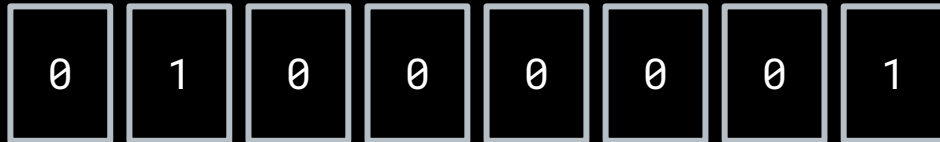
two nor gates, wired into each other



reset = 1: q becomes 0. and stays.

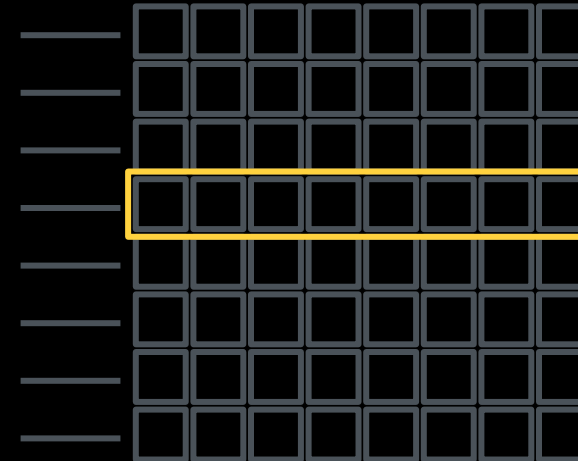
from one bit to memory

eight of them: a register, one byte



= 65 = A, until you overwrite it

millions of them, each with an address: ram



address lines pick one row, data lines read or write it

as long as the power is on, the feedback keeps every bit alive.  
switch off, and the loop collapses: that is why ram forgets.

storing and processing are the same switches, wired differently.



adding

1 + 1 in binary

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10$$

two result digits: the sum and the carry

which gates produce these two digits?

the half adder: two gates

|   |   | xor |
|---|---|-----|
| a | b | sum |
| 0 | 0 | 0   |
| 0 | 1 | 1   |
| 1 | 0 | 1   |
| 1 | 1 | 0   |

the sum is 1 when the bits differ: xor.

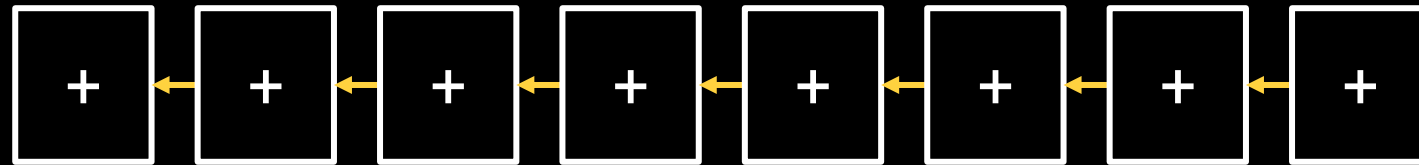
the half adder: two gates

| a b |   | xor<br>sum | and<br>carry |
|-----|---|------------|--------------|
| 0   | 0 | 0          | 0            |
| 0   | 1 | 1          | 0            |
| 1   | 0 | 1          | 0            |
| 1   | 1 | 0          | 1            |

two gates, and the machine adds two bits.

eight of them in a row: a byte adder

$$\begin{array}{r} 178 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 \quad 1 \quad 0 \\ + 40 \quad 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \end{array}$$



full adder:  
about five gates each

carry travels right to left

$$= 218 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0$$

eight adders in a row add two bytes. remember  $178 + 40$ .

the rest of arithmetic is built the same way

## subtract

add the negative

same adder, one trick with  
not

## multiply

shift and add

times two: one position to the  
left

## compare

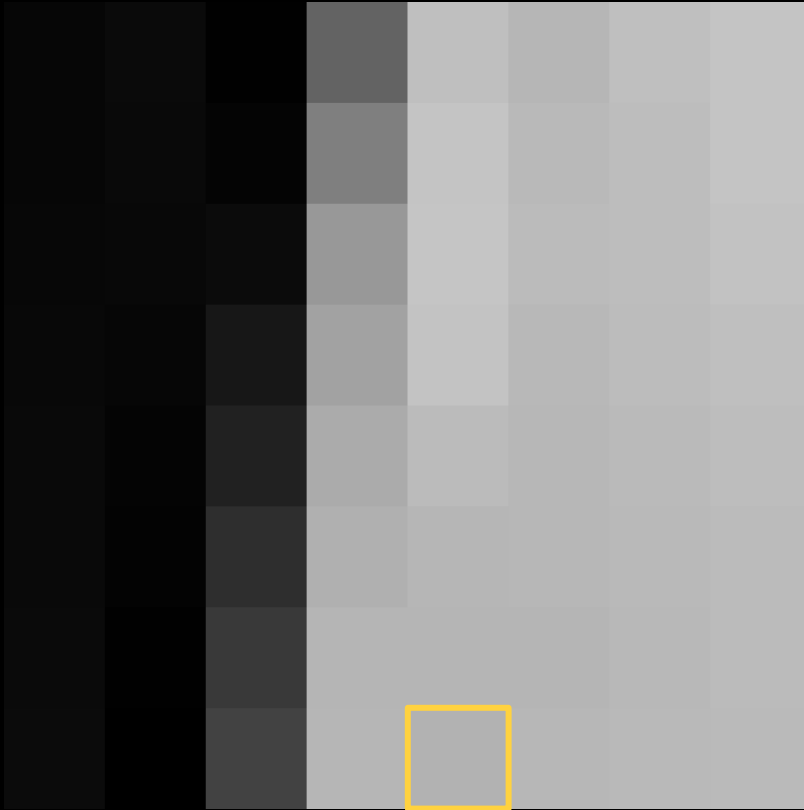
subtract, look at the sign

is a greater than b?

everything lands on the adder, and the adder lands on four gates.

a photo, pixel by pixel

a photo is a grid of bytes



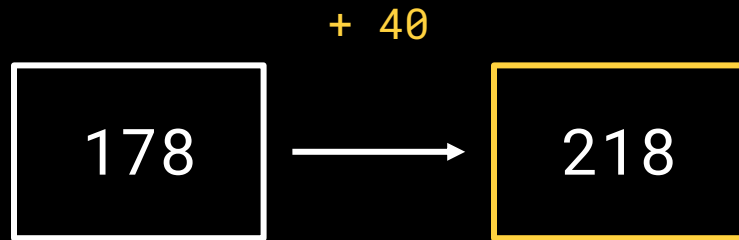
|    |    |    |     |     |     |     |     |
|----|----|----|-----|-----|-----|-----|-----|
| 6  | 10 | 1  | 99  | 191 | 182 | 191 | 196 |
| 6  | 9  | 4  | 127 | 196 | 185 | 189 | 196 |
| 7  | 8  | 11 | 152 | 197 | 187 | 189 | 194 |
| 8  | 6  | 23 | 162 | 195 | 184 | 188 | 191 |
| 9  | 4  | 33 | 171 | 187 | 183 | 186 | 189 |
| 9  | 3  | 46 | 176 | 182 | 183 | 185 | 187 |
| 10 | 1  | 57 | 181 | 181 | 181 | 184 | 187 |
| 11 | 0  | 66 | 182 | 178 | 183 | 185 | 186 |

256 × 256 pixels = 65 536 bytes. 0 is black, 255 is white.

the computer does not see a parrot. it sees bytes.



brighter: add 40 to every pixel



1 0 1 1 0 0 1 0

0 0 1 0 1 0 0 0

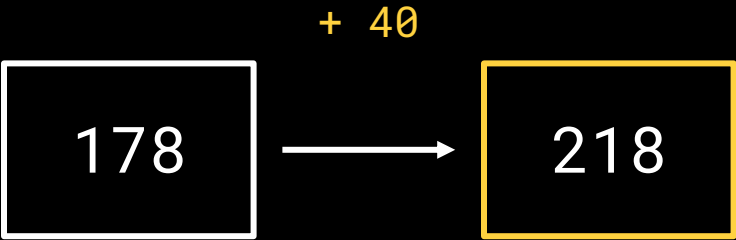
---

1 1 0 1 1 0 1 0

the same adder as before

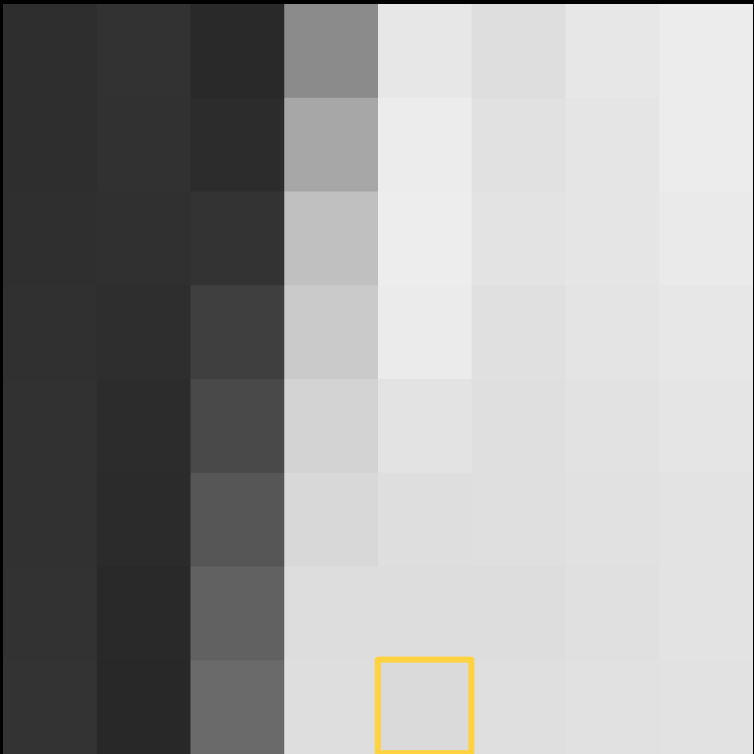
„brighter“ is: add 40 to a byte.

brighter: add 40 to every pixel



1 0 1 1 0 0 1 0  
0 0 1 0 1 0 0 0  
-----  
1 1 0 1 1 0 1 0

the same adder as before



and the next pixel,  
and the next.

65 536 times.

no step knows anything about the picture. the same sum, very often.

what happens at 255?

$$230 + 40 = 270$$

1 0 0 0 0 1 1 1 0

nine bits. the byte has eight.

wrap

14

the ninth carry is dropped

clamp

255

compare first, then cap at 255



wrap: black speckles in the sky



clamp: bright stays white

the adder has no idea what 255 means. you have to tell it.

more filters, same toolbox

darker

$\text{value} - 40$

subtract

invert

$255 - \text{value}$

that is not, on every  
bit

black & white

$\text{value} > 128 ?$   
 $255 : 0$

compare

blend two  
photos

$(a + b) / 2$

add, then shift right

every filter in your photo app is of this kind. just more arithmetic.

try it



## the pixel filter

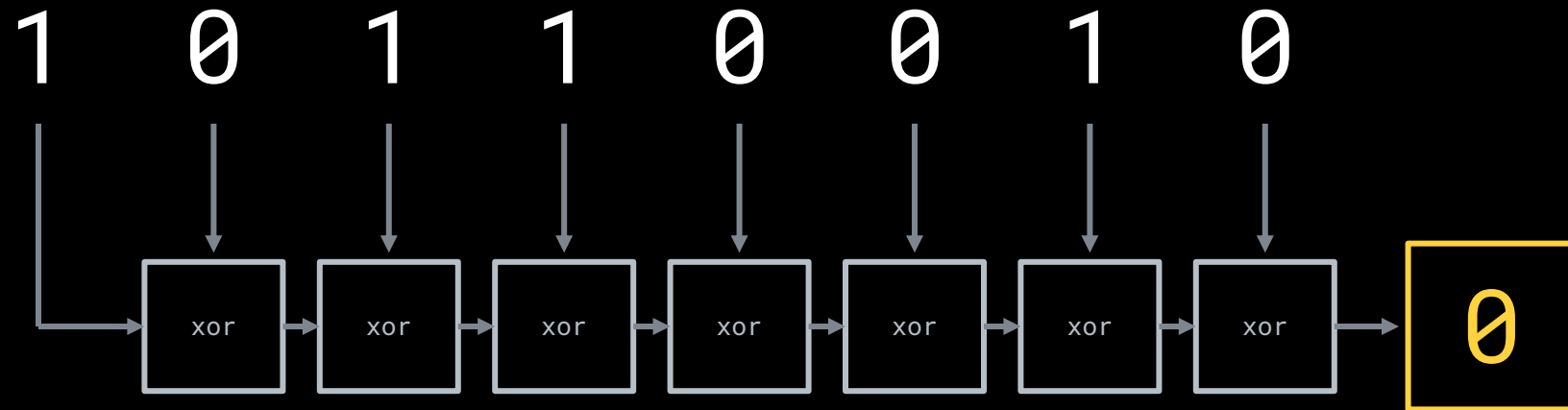
one pixel at a time, then all of them.  
watch the counter. switch clamp to wrap.

[winf-hsos.github.io/lifi-concept-demos/pixel-filter](http://winf-hsos.github.io/lifi-concept-demos/pixel-filter)

the computer is not clever. it is fast and stubborn.

one gate, two jobs

xor over a byte: a check bit



4 ones, an even number: the check bit is 0.

flip any single bit on the way, and the count turns odd.

xor over a byte answers one question: even or odd?

xor with a key: encryption

|         |   |   |   |   |   |   |   |   |
|---------|---|---|---|---|---|---|---|---|
| message | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 1 |
| key     | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 |
| secret  | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |



xor with a key: encryption

|         |   |   |   |   |   |   |   |   |
|---------|---|---|---|---|---|---|---|---|
| message | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 1 |
|---------|---|---|---|---|---|---|---|---|

|     |   |   |   |   |   |   |   |   |
|-----|---|---|---|---|---|---|---|---|
| key | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 |
|-----|---|---|---|---|---|---|---|---|

|        |   |   |   |   |   |   |   |   |
|--------|---|---|---|---|---|---|---|---|
| secret | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |
|--------|---|---|---|---|---|---|---|---|

|        |   |   |   |   |   |   |   |   |
|--------|---|---|---|---|---|---|---|---|
| secret | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |
|--------|---|---|---|---|---|---|---|---|

|     |   |   |   |   |   |   |   |   |
|-----|---|---|---|---|---|---|---|---|
| key | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 |
|-----|---|---|---|---|---|---|---|---|

|         |   |   |   |   |   |   |   |   |
|---------|---|---|---|---|---|---|---|---|
| message | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 1 |
|---------|---|---|---|---|---|---|---|---|

four gates, a few billion times a second.  
that is all.